

모방학습과 인간 피드백 (Imitation Learning & RLHF)

행동 복제와 DAgger · 선호 기반 보상모델 · 모방학습 vs 강화학습 선택

Machine Learning 2 · 12주차

한국과학영재학교 (KSA)

Chapter 12

이번 주차 개요

이 챕터를 마치면

- 행동 복제(BC)를 “ (s, a) 시연 쌍을 라벨로 쓰는 지도학습”으로 설명하고, 학습·배포 분포 어긋남으로 오차가 눈덩이치는 **복합 오차**의 본질을 짚을 수 있다.
- **DAgger** 네 줄 — 에이전트가 자기 정책으로 달리고, 전문가가 방문한 상태에 라벨만 답하는 반복 — 을 장난감 환경에 적용해 본다.
- **Bradley-Terry** 선호 쌍에서 보상모델을 학습해 PPO 보상으로 쓰는 RLHF 두 단계를 설명한다.
- 가진 자원(시연·비교·보상함수)과 p^T 감쇠로 BC / DAgger / 선호기반 / BC+RL 선택을 정량적으로 따질 수 있다.

12.1 시연에서 — BC의 지름길과 한계, DAgger.

12.2 비교에서 — Bradley-Terry 보상모델과 RLHF.

12.3 무엇을 쓸 것인가 — 원칙이 아니라 숫자로.

12.1 모방학습: 행동 복제와 DAgger

시행착오가 위험하거나 비싼 문제

- Ch. 2~11의 RL: 에이전트가 스스로 시행착오를 겪으며 **보상 신호**로 배웠다.
- 그런데 어떤 문제는 **시행착오 자체가 위험하거나 비싸다**.
- 예: 로봇 팔이 사람 옆에서 물건을 옮기는 법을 **무작위로 시도**하며 배우게 둘 수는 없다.
- 예: 자율주행 — “사고를 내며 배우는” 학습은 허용 안 됨.

이번 장의 두 가지 방법

보상을 직접 최적화하는 대신, **사람의 시연**이나 **사람의 선호**로부터 배운다 — 모방학습과 RLHF.

RL의 두 전제와, 그 전제가 무너지는 문제

Ch. 2~11 모든 RL 알고리즘의 **두 가지 전제**

- ① **보상함수가 코드로** 쓰여 있어야 한다 — “무엇이 좋은지”를 수식으로 정의할 수 있어야.
- ② **에이전트가 환경을 자유롭게** 돌아다녀야 한다 — 나쁜 행동을 골라보며 대가를 겪어야 배운다(탐험-활용).

두 전제가 동시에 무너지는 문제가 실제로 존재

- 로봇 팔: (1) “자연스러운 운동”을 어떤 스칼라 함수로 쓸지 모르겠고
- (2) 무작위 탐험이 사람을 다치게 할 수 있다.
- 자율주행: (2)가 극단적.

행동 복제(BC): 시연 데이터를 지도학습에 꽂아 넣기

- 모방학습의 가장 단순한 형태 = **행동 복제** (Behavior Cloning, BC).
- 전문가(사람 또는 기존 정책)가 남긴 (s, a) **시연 데이터**를 모아서 “이 상태에선 이 행동”이라는 **지도학습 문제**로 그대로 바꾼다.
- 보상함수 설계도, 탐험도 불필요.
- 상태 → 행동을 출력하도록 학습 — 이산이면 **분류**, 연속이면 **회귀**.

이미 잘하는 존재(사람 운전자, 사람이 조종한 로봇, 오래된 제어기)가 남긴 기록이 가장 가치 있는 자원인 문제.

BC 코드: 로지스틱회귀와 한 줄도 다름없다

```
import math

def sigmoid(z):
    return 1 / (1 + math.exp(-max(-20, min(20, z))))

def train_behavior_cloning(demos, epochs, lr):
    # demos: [(state, action), ...] 이진( 예시, action = 0 또는 1)
    w, b = 0.0, 0.0
    for _ in range(epochs):
        for s, a in demos:
            pred = sigmoid(w * s + b)
            grad = pred - a # 로지스틱회귀와 똑같은 형태
            w -= lr * grad * s
            b -= lr * grad
    return w, b
```

$demos = (x, y)$ 쌍, $\text{sigmoid} = \text{모델}$, $\text{grad} = \text{pred} - a = \text{교차엔트로피 그래디언트}$ — 갱신 규칙 그대로.

새 알고리즘이 아니라, 데이터 출처만 바꾼 것

Ch. 2 로지스틱회귀 코드와 나란히 보면 **한 줄도 바뀌지 않았다.**

- BC는 “새로운 알고리즘”이 아니라 기존 지도학습에 **데이터 출처(전문가 시연)**를 꽂아 넣은 것.
- Q-learning은 정답을 TD 오차로 **스스로** 만들 (Ch. 6). BC는 사람이 만든 (s, a) 쌍을 **그대로** 정답으로 씀.

이 절의 스토리, 한 문장으로

시연 데이터의 분포가 **배포 시 상태 분포**와 어긋나기 시작하면, 그 지름길이 어떻게 무너지는가, 그리고 DAgger가 그 어긋남을 어떻게 다시 맞추는가.

역사적 배경: “원격 조종”에서 “시연”으로

- 뿌리는 실용적 — 1980년대 말 스탠퍼드 **AVI** (Action-Vision-Inference) 프로젝트.
- 카메라로 도로를 보고 “이 장면에서 핸들을 이렇게 돌렸다”는 **사람 운전자 기록**을 모아서 차 조종을 학습.
- 현대 모방학습의 구조 — “(시각 입력, 조향) 쌍을 모아서 분류” — 가 그때 이미 서 있었다.
- 그러나 곧 한계: 사람이 보여준 그 장면에겐 잘했지만, 자신의 작은 실수로 빗나간 낯선 장면에 부딪히면 차가 멈춰 서거나 위험한 행동을 했다.

이 실패 패턴이 2011년에야 Ross, Gordon & Bagnell에 의해 **복합 오차**(compounding error)로 이론화됐고, 같은 논문이 해법 **DAgger**를 제시했다.

손으로 풀 수 있는 장난감 환경: 3×4 회랑

	열0	열1	열2	열3
0	(0,0)	(0,1)	(0,2)	[목표]
1	(1,0)	[위험]	[위험]	(1,3)
2	[시작]	(2,1)	(2,2)	(2,3)

- 3줄 $\times$ 4열. 시작 (2, 0), 목표 (0, 3).
- (1, 1), (1, 2)는 “공사 중” — 들어오면 **즉시 실패**.
- 4방 이동, 벽에 부딪히면 **제자리**.
- 전문가 = “안전한 최단 경로 알고리즘”.

3×4 회랑: 시연 경로(파랑)와 시연 밖 상태. 초록=목표, 빨강=위험칸, 주황=시연에 없던 시작 — BC의 기본 행동 'right'가 벽에 막혀 제자리. Dagger는 (1,3)에서 전문가의 'up'을 배워 탈출



초록=목표, 빨강=위험칸. 주황 (1, 3)은 시연에 없던 시작 — BC의 기본값 right가 벽에 막힌다.

전문가 시연 5개: BC 정책은 조회 테이블

상태	전문가의 행동
(2,0)	up
(1,0)	up
(0,0)	right
(0,1)	right
(0,2)	right

- 전문가 시연 = 하단에서 바로 위로 올라와 상단 회랑을 따라가는 **5스텝 직진 경로**
 $(2, 0) \rightarrow (1, 0) \rightarrow (0, 0) \rightarrow (0, 1) \rightarrow (0, 2) \rightarrow (0, 3)$.
- $(1, 1)/(1, 2)$ 공사 구간을 **완전히 피하는** 구조적 우회.
- **BC 정책** = `table.get(s, "right")` — 시연에 없던 상태엔 **시연의 다수결(기본값)**

배포 실패: “시물레이션에선 돌아갔는데”

시작 (2, 0) — 시연 경로를 그대로 따감:

(2, 0) up → (1, 0) up → (0, 0) right → (0, 1) right → (0, 2) right → (0, 3)

목표 도착 (5스텝) — 학습 데이터 위에서 완벽하다.

시작 (1, 3)으로 변경: 시연에 **한 번도 등장 하지 않은** 상태. 기본값 right인데, (1, 3)에서 right는 **벽**이다.

시작이 2칸만 달라졌을 때. 핵심: “시연 데이터가 적어서”가 아니라 **시연이 커버하지 못한 상태 분포** 문제.

(1, 3) right → (1, 3) right → (1, 3) right → ...

벽에 부딪히며 제자리를 돌다가 타임아웃.

같은 패턴: (2, 2)도 기본값 right로 (2, 3)까지 갔다
벽에 막혀 죽는다.

문제 본질을 수식으로: 학습 오차 vs 배포 오차

- BC의 학습 오차와 배포 오차는 **서로 다른 분포 위의** 오차.
- 학습 오차 — 전문가 정책 π^E 의 상태 분포 d_{π^E} 위에서:

$$\text{학습 오차} = \mathbb{E}_{s \sim d_{\pi^E}} [L(s)] \approx \frac{1}{n} \sum_{i=1}^n L(s_i)$$

- 배포 오차 — 배포 시 정책의 상태 분포 d_{π} 위에서(진짜 원하는 것):

$$\text{배포 오차} = \mathbb{E}_{s \sim d_{\pi}} [L(s)]$$

핵심

“학습 오차를 최소화하라”는 원칙은 $d_{\pi} \approx d_{\pi^E}$ 일 때만 배포 오차 최소화와 일치. **복합 오차**란 바로 이 두 분포의 어긋남. 기계학습 언어로는 **공분포 시프트**(covariate shift).

왜 BC만으로는 부족한가: 복합 오차의 눈덩이

- 학습은 **전문가가 실제로 방문한 상태들**에서만 이뤄진다.
- 배포된 정책은 작은 실수 하나로 **전문가가 가본 적 없는** 상태로 빗나갈 수 있다.
- 낯선 상태에선 정책을 **배운 적 없으므로** 더 큰 실수.
- 그 실수가 또 다른 낯선 상태로 이어진다.

(compounding error)

오차가 시간이 지날수록 눈덩이처럼 불어나는 문제.

어긋남은 한 스텝의 실수($\pi \neq \pi^E$ 인 스텝)가 경로를 시연 밖으로 빼내는 순간 시작된다.

DAgger: 정책이 달리고, 전문가가 답만 한다

```
D = dict(demo) # ①원래시연데이터로시작
for rnd in range(rounds):
    s = s0
    while not terminated(s):
        a = policy_from(D)(s) # ②에이전트가자기 ' 정책으로 ' 진행
        D[s] = expert(s) # ③전문가가방문한 ' 상태에 ' 라벨만
        s = step(s, a) # 확률적 ( 교란은여기서 )
    # ④ D 전체로정책을다시학습
```

(1) 학습된 정책으로 직접 환경에 돌아본다 → (2) 마주친 상태에 전문가에게 다시 물어본다 → (3) 새 데이터를 원래 시연에 합쳐 재학습. 반복.

네 줄의 각 — 역할 분담이 DAgger의 정체

②가 전문가 정책으로 간다면

- = “시연을 더 많이 모으는 것”과 같음
- 위에서 본 바와 같이 **도움이 안 됨**

③이 다른 상태에 라벨을 찍는다면

- 배포 시 마주칠 확률이 **0인 상태**에 시간을 쓰는 것

정체

에이전트가 운전하고, 전문가는 내비게이션만 해주는 구조.

- 배포 오차 $\mathbb{E}_{s \sim d_\pi} L(s)$ 를 줄이려면 학습 데이터가 d_π 위에 놓여야 한다.
- π 는 매 라운드 바뀐다 — DAgger는 “달리는 말 위에서” 이 문제를 **반복으로 안정화**.

DAgger 손으로 돌리기: 2라운드 복귀

시작 (1, 3), 시연 테이블 5개(아직 (1, 3) 라벨 없음), 라운드당 최대 5스텝.

라운드	에이전트 경로(자신의 정책)	전문가의 라벨	추가	결과
1	$(1, 3) \rightarrow (1, 3) \rightarrow (1, 3) \rightarrow (1, 3) \rightarrow (1, 3)$ — 기본값 right가 벽에 막혀 제자리	$(1, 3) \rightarrow$ up		타임아웃
2	$(1, 3) \text{ up} \rightarrow (0, 3)$	$(1, 3) \rightarrow$ up (이미 존재)		목표, 1스텝

5번의 “잘못”이 바로 데이터를 만든다

시연 5개 $\rightarrow$ 6개. **한 칸의 라벨**만 늘렸는데, (1, 3)에서의 배포 오차가 완전히 0.

“시연 1000개 더”로는 안 되는 이유

- 전문가가 (2, 1), (2, 2), (2, 3)에서 시작해 1000개 시연해도, 경로들은 모두 “(2,?)에서 up, 상단 회랑 right...”의 **변주일 뿐**.
- **전문가 자신이 방문하지 않는 (1, 3)의 라벨은 절대 생기지 않는다.**

핵심 구분

시연의 **수량**이 아니라, **시연이 어떤 상태 분포 위에 놓여 있는가**가 문제.

Dagger와 같은 정신: “이론상 이상적인 상황이 아니라, **실제로 마주칠 상황**을 기준으로 학습한다” (Ch. 6 SARSA와 비슷한 관점).

복합 오차를 숫자로: p^T 문제

- 배포 시 매 스텝마다 “전문가와 같은” 행동을 할 확률을 $p < 1$ 이라 하면, T 스텝 episode에서 **한 번도 오점을 지나치지 않을 확률** = p^T .
- $p = 0.99$ — “매 스텝 99% 전문가를 따라” 좋은 BC 수준 — 이라도 계산해보자.

episode 길이 T	50	100	200	300
$p = 0.980$	0.364	0.133	0.018	0.002
$p = 0.990$	0.605	0.366	0.134	0.049
$p = 0.995$	0.778	0.606	0.367	0.222

$p = 0.99$ 를 써도 300스텝에서 “한 번도 빠뜨리지 않는” 확률은 4.9%.

p^T 감쇠: 완주 가능 길이는 어떻게 결정되는가

- “완주 확률이 50%”가 되는 episode 길이:

$$T = \frac{\log(0.5)}{\log(p)}$$

- $p = 0.99$ 이면 **약 69스텝**, $p = 0.995$ 이면 **약 138스텝**.
- 1스텝 성공률을 0.5%p만 올려도 완주 가능한 길이가 **두 배**.

읽는 법

긴 episode에서는 “거의 불가능”과 “가끔 된다”를 가르는 것이 1스텝 성공률의 **0.5%p**다.
12.3절에서 이 표를 5×5 격자 실험과 대조한다.

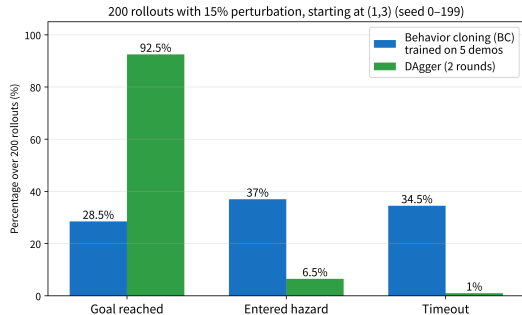
교란 실험: 세계가 움직이면 둘은 갈라진다

- 장난감 환경은 **결정적**이라 BC 실패가 “시연 밖 상태의 존재”만으로 설명됐다.
- 실제 배포 환경은 **확률적** — 센서 노이즈, 다른 차량, 표면 마찰.
- 매 스텝 **15% 확률로 명령과 다른 이동**으로 바뀌는 교란을 넣고, 시연에 없던 시작 (1,3)에서 **200회** 돌아옴.

```
def step_noise(s, a, rng, p=0.15):  
    if rng.random() < p:          # 15%: 명령이 다른 이동으로 바뀐다  
        a = rng.choice([x for x in ("up", "down", "left", "right") if x != a])  
    return step(s, a)
```

교란 실험 결과: 숫자가 말하는 것

결과 (200회)	BC(시연 5개)	DAgger(2라운드)
목표 도달	57회 (28.5%)	185회 (92.5%)
위험칸 진입	74회 (37%)	13회 (6.5%)
미도달(타임아웃)	69회 (34.5%)	2회 (1%)



시작 (1, 3)에서 200회 롤아웃 — BC vs DAgger

- **BC의 37%가 위험칸 진입** — 시연엔 위험칸이 하나도 없는데, 기본값 right로 시연 밖에서 배회하다 공사 구간에 밀려든다.
- “시연이 안전했으니 정책도 안전할 것” — **시연 분포와 배포 분포가 일치할 때만** 성립.

DAgger의 보장: 무엇인가, 무엇 아닌가

- “복합 오차를 해결한다”고 읽으면 안 된다. 정확히는 학습 데이터 분포를 d_{π^E} 에서 **정책 스스로의 d_π 쪽으로** 계속 끌고 간다.
- Ross et al. (2011)의 양적 결과:
- BC의 기대 에러: episode 길이 H 에 대해 최악 H^3 **급** (세제곱)
- DAgger: 반복을 늘리면 H **에 선형**으로만 커진다.

실전 해석

“세제곱 $\rightarrow$ 선형” = “긴 episode에서 도저히 못 쓴다 $\rightarrow$ 쓸만해진다”. 12.3절의 p^T 감쇠와 같은 현상의 **이론적 뒷편**.

DAgger도 못 하는 것 (3가지)

- ① **전문가 라벨 비용이 실존**: 전문가가 사람(실물 로봇)이면 매 라운드 비싸지고, 반복 횟수 자체가 하이퍼파라미터.
- ② **아직 방문하지 못한 상태**는 커버 안 됨 — 교란 확률이 극단적으로 낮은 상태, 매우 긴 episode의 후반부. (실험의 “13회”가 그것.)
- ③ **전문가가 최적 정책이 아니면** DAgger도 **전문가의 상한을 넘지 못한다** — 12.3절 실험의 “전문가가 10스텝 우회로 간다”가 정확히 이 케이스.

그 때 필요한 것은 DAgger가 아니라 12.3절의 BC+RL 하이브리드(또는 12.2절의 선호 기반).

실전 체크리스트: 전문가 비용, 데이터 가중, GAIL

- ① **전문가 비용** — 시뮬레이션에선 “전문가”가 이미 잘 동작하는 기존 컨트롤러가 될 수 있음 $\Rightarrow$ 호출 즉시 답이므로 DAgger가 **완전히 자동화**. 실물 로봇(사람 매 라운드)이면 라운드를 작게(2~5회) 잡고, 나머지는 BC+RL(12.3절)로 메운다.
- ② **데이터 가중** — 라운드가 쌓이면 D 는 초기 시연과 최근 라벨의 혼합. 최신 라벨에 가중을 크게 주면(예: 2배/라운드) 현재 분포에 빨리 맞춰지되 초기 정보를 빨리 잊음 — **탐색-활용의 또 다른 얼굴**.
- ③ **전문가가 “비교”만 줄 수 있다면** — “(s,a)” 대신 “두 시도 중 저쪽이 낫다”라면서 DAgger의 ③을 Bradley-Terry 손실로 대체: **GAIL** (Generative Adversarial Imitation Learning) 계열 — 12.2절이 그 변주.

자주 하는 실수: “시연 부족”을 “수량 부족”으로

- (1, 3) 예제의 가장 흔한 반응: “전문가가 다른 시작점에서 시연 500개 더 보여주면 되잖아.”
안 된다.
- (2, 1), (2, 2), (2, 3) 시작 시연은 전부 “(2,?) $\rightarrow$ up $\rightarrow$ (0,?) $\rightarrow$ right ...”의 변주.
- (1, 3)은 아무도 시연 경로 위에서 지나가지 않는 상태 — 500개, 5000개까지 늘려도 라벨 한 개도 안 생김.

판정 기준: 수량이 아니라 분포

“배포 시 **시연에 없던 상태**를 방문할 확률이 몇 %인가?” 0에 가까우면 BC로 충분. 아니면(긴 episode / 교란 큰 환경 / 전문가가 비최적 경로 시연) DAgger 또는 BC+RL.

FAQ: DAgger 13회, 1000개 시연, 셋 비교

- **Q: 교란 실험에서 DAgger도 위험칸에 13번 — 해결이면 0이 되어야?** A: 해결이 아니라 **완화**. 15% 교란은 5스텝 샘플링에서 지나가지 않은 단일 사건(예: (1, 3)에서 'left'로 (1, 2) 직행)을 남김. 13회 = **방문 기반 라벨링의 본질적 한계** — BC+RL(12.3)이 남은 이유.
- **Q: 시연 1000개 모으는 게 더 간단하지 않나?** A: 누가 달리는가가 분포를 결정 — 시연 1000개는 전문가 정책의 경로. 학생 정책의 “(1,3) 벽 막힘” 패턴은 전문가가 한 번도 빠지지 않으므로 라벨 없음. **비용**도 학생이 방문한 상태 수만큼의 라벨이 훨씬 적음.
- **Q: DAgger(12.1) / 선호 기반(12.2) / BC+RL(12.3) 정리?** A: 사람이 주는 것이 다르면 알고리즘이 다름 — (상태, 정답 행동)→DAgger(상한=전문가); (두 시도 중 더 나은 쪽)→선호 기반 보상모델(PPO로 **전문가 초월 가능**); 보상함수만 →순수 RL.

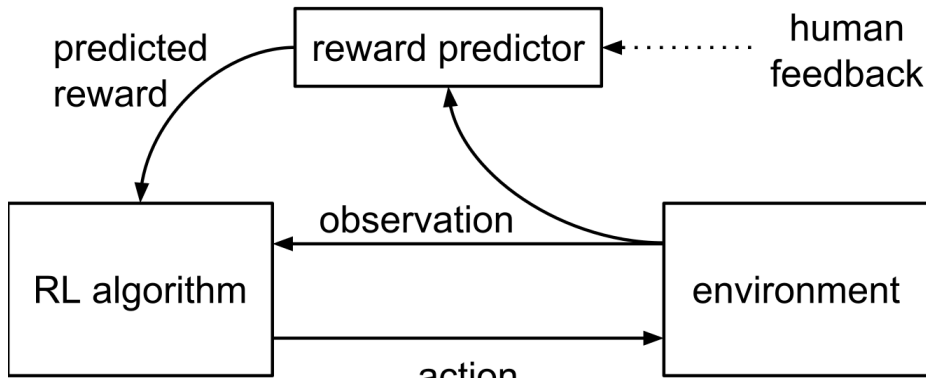
12.1 정리

- BC = 시연 (s, a) 쌍을 라벨로 쓴 **지도학습**. 로지스틱회귀 코드 한 줄도 안 바뀜.
- 실패는 **수량**이 아니라 **분포**: 학습 분포 d_{π^E} vs 배포 분포 d_{π} (공분포 시프트, 복합 오차).
- DAgger: “에이전트가 자기 정책으로 달리고, 전문가가 방문한 상태에 라벨만 답” 반복 — 2라운드만에 $(1, 3)$ 타임아웃을 1스텝 완주로, 15% 교란 환경 위험칸 $74 \rightarrow 13$ 회.
- 보장은 “ $H^3 \rightarrow$ 선형”이지 해결이 아님. 방문 못 한 상태와 비최적 전문가의 상한은 못 넘음.

12.2 선호 기반 보상모델과 실습

시연이 어려운 문제에는 “비교”가 통한다

- 예: “이 로봇 걸음걸이가 더 자연스럽다” — 정답 하나를 꼭 집어 **시연하기는 어렵지만**, 두 걸음걸이를 보고 **어느 쪽이 나은지** 고르기만 하면 됨.
- 사람이 매번 “정답 행동”을 알려주는 대신, 로봇이 만든 **두 시도(궤적)** 중 더 나은 쪽을 **고르기만**.
- 이 아이디어는 형태만 바뀌었을 뿐, LLM을 다듬는 **RLHF**(Reinforcement Learning from Human Feedback, Christiano et al., 2017)와 **정확히 같은 구조**.



Bradley-Terry 모델: 선호 확률을 시그모이드로

- 같은 상황 x 에서 두 궤적 y_w (더 선호), y_l (덜 선호).
- 보상모델 $r_\phi(x, y)$ 가 y_w 에 더 높은 점수를 주도록 학습.
- **브래들리-테리 모델**: 선호 확률을 시그모이드로 모델링:

$$P(y_w \succ y_l \mid x) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$$

- 이 확률을 최대화하는 손실은 다시 **로지스틱회귀와 같은 교차엔트로피 형태**:

$$-\log \sigma(r_w - r_l)$$

단계 2

학습된 r_ϕ 의 점수를 PPO(Ch. 9~11)의 **보상으로 그대로** 써 정책을 학습.

선호 기반 학습도 결국 로지스틱회귀

```
def reward_model_loss(r_win, r_lose):  
    # Bradley-Terry:  $P(y_w > y_l) = \text{sigmoid}(r_{\text{win}} - r_{\text{lose}})$   
    return -math.log(sigmoid(r_win - r_lose))
```

- BC(12.1)의 $\text{grad} = \text{pred} - a$, 여기서 “win이 이겼다”가 정답 라벨.
- **패턴의 반복:** 로지스틱회귀 $\rightarrow$ BC $\rightarrow$ Bradley-Terry 보상모델. 구조는 늘 “시그모이드 + 교차엔트로피”.

왜 “시연”이 아니라 “비교”인가 — 두 가지 이유

첫째, 시연 자체가 어려운 문제가 있다

- “더 자연스러운 걸음걸이”, “더 안전한 주행”, “더 정중한 말투”
- 최적 하나를 정밀하게 만드는 게 아니라 주어진 두 가지 중 나은 쪽을 가리는 데 인간의 직감이 훨씬 잘 맞음.
- “비교는 쉬우나 절대 정의는 어렵다” — 연속적인 질을 이산 판정(2택1)으로 줄이면 사람이 **일관되게** 라벨을 댈 수 있음.

둘째(더 근본적): 시연은 상한이 사람

- BC는 전문가가 보여주지 않은 전략을 **절대** 만들어낼 수 없다.
- 선호 기반 보상모델은 “더 나은 쪽”만 알리면, **사람이 시연한 적 없는 새 시도**에도 점수를 줄 수 있다.
- PPO로 최적화하면 사람이 보여준 적 없는 행동 패턴을 **발견**할 수 있다.

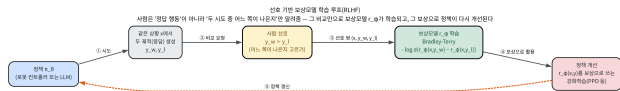
시연: 따라잡기 vs 비교: 넘어서기

- 모방학습 = “전문가를 **따라잡는**” 지름길.
- 선호 기반 보상모델 = “비교만 받아들이면서도 전문가를 **넘어서게 할 수 있는**” 우회로.
- 12.3절 표의 “**성능 상한**” 행이 이 둘을 가른다.

구분해서 이해할 것

“사람이 **뭘 하는가**”(비교) — 여전히 **지도학습** (교차엔트로피). **알고리즘이 그걸 어떻게 쓰느냐** (비교를 보상함수로 변환, PPO에 최적화 대상으로 넘김) — 이 절이 새로운 부분.

RLHF의 두 단계 전체 그림



- ① 정책이 같은 x 에서 y_w, y_l 생성 → ② 사람에게 표시 → ③ 선호 쌍 (x, y_w, y_l) 수집 → ④ Bradley-Terry로 r_ϕ 학습 → ⑤ r_ϕ 를 보상으로 PPO로 π_θ 갱신 → ① 반복.

단계 1: 보상모델 학습(지도학습)

- 선호 쌍 (x, y_w, y_l) 으로 r_ϕ 학습.
- Ch. 2 같은 **정지** 문제 — 탐색도, 환경 상호작용도 없음.
- LLM RLHF 실전: 사람 비교 응답 수만 ~ 수십만 개 수집.

단계 2: 보상 최적화(강화학습)

- $r_\phi(x, y)$ 를 **보상으로 그대로** PPO에.
- “보상이 사람”이 아니라 “사람의 선호를 학습한 **대리 모델**”.

LLM RLHF: 두 단계 파이프라인이 표준

Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.



The policy generates an output.



The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



- “사람의 선호로 보상모델을 학습하고, 그 보상을 PPO에 넣어 정책을 최적화” — LLM 챗봇 훈련의 **표준**.

- **실용적 이유: 주기가 다름**

- 비교 수집 = 비싸고 느리고, 사람 오프라인 작업.

- **최적화** = 싸고 빠르고, 자동화 RL.

- 선호 쌍 한 번 모아두면, 단계 2는 **사람 없이** 반복 가능.

InstructGPT (Ouyang et al., 2022) 3단계 파이프라인: SFT → 보상모델(RM) 학습 → PPO 강화학습.

보상모델을 PPO 보상으로 넣으면

- PPO 입장에서는 **아무것도 바뀌지 않음**.
- Ch. 11.2: PPO는 “스텝마다 환경에서 오는 스칼라 보상”을 전제로 — 그 스칼라를 “환경 값” 대신 “보상모델이 점수 매긴 값 $r_\phi(x_t, y_t)$ ”로 교체할 뿐.
- 정책 그래디언트·어드밴티지·클리핑 등 내부 수식 **그대로 재사용** — **보상의 출처만 바뀐**.

기억할 것

선호 기반 학습이 강화학습을 **대체하는** 것이 아니라, 그 강화학습에 넣을 **보상을 사람이 설계하는 대신 사람의 비교로부터 학습하는** 방법.

그리고 그 함정: 보상 해킹과 Goodhart

- PPO는 r_ϕ 를 **완벽한** 것으로 믿고 값을 높이는 방향으로 움직임.
- 그런데 r_ϕ 는 결국 **학습된 근사**.
- “사람의 진짜 선호”와 “모델이 추정한 선호”가 다른 영역으로 벗어나면:

(reward hacking)

정책이 **보상모델의 약점(잘 못 점수 매긴 지역)을 파고들며** 점수는 오르는 데 실제 선호는 안 좋아지는 행동을 배울 수 있다.

- **Goodhart의 법칙** — “측정 지표가 목적이 되면 더 이상 좋은 지표가 아니다” — 의 **강화학습 버전**.
- 보상을 “진짜 선호”가 아니라 “모델 출력”으로 대치한 순간, 최적화 대상은 **모델 출력을 높이는 것**이 됨.

보상 해킹의 모습: 로봇 vs LLM

로봇 제어 (Ch. 13)

- 실제 보상을 회피하는 **다른 지표**를 극대화
— 예: 벽에 부딪히는 횟수를 최소화하려
벽에서 멀리만 맴도는.

LLM RLHF

- 보상모델이 좋아하는 **형식**(과장, 장황함)에
맞춰 **실제 내용은 퇴보.**

이번 절의 범위

이 위험을 **인지**하는 수준에서 멈춘다. Chapter 13에서 실제 로봇에서 어떻게 관찰되는지 다시 본다.

실습: 선호 쌍만으로 숨겨진 보상함수 복원

- 로봇 걸음걸이를 “(부드러움, 덜컹거림)” 두 특징으로 요약.
- 사람은 (걸에 드러나지 않는) 진짜 보상

$$r^*(\text{traj}) = 2 \times \text{부드러움} - 1 \times \text{덜컹거림}$$

을 마음속으로 갖고 선호 표현.

- 알고리즘은 r^* 를 **전혀 모름** — 비교 데이터 **300개**만 봄.

```
for epoch in range(200):  
    for win, lose in pairs:      # 선호된(, 덜선호된 ) 궤적쌍  
        rw, rl = r_phi(win, w1, w2, b), r_phi(lose, w1, w2, b)  
        p = sigmoid(rw - rl)      # Bradley-Terry 예측  
        grad = p - 1             # 정답 = 항상 "이win 이겼다"(=1)  
        w1 -= lr * grad * (win[0] - lose[0])  
        w2 -= lr * grad * (win[1] - lose[1])
```

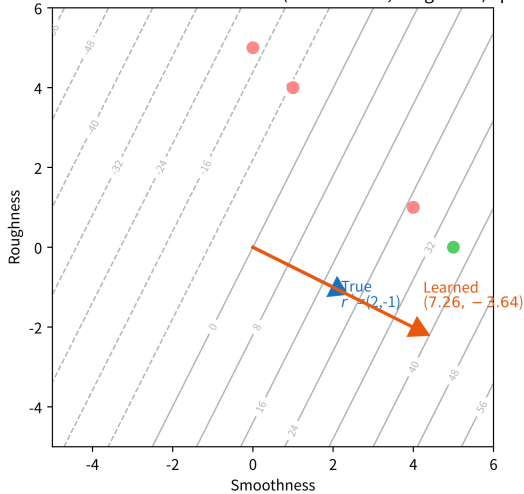
실습 결과: 복원된 것 — 크기가 아니라 방향

200 에폭 학습 후 (seed 42):

	w1(부드러움)	w2(덜컹거림)
학습된	7.264	-3.643
진짜	2	-1

- 비율 $w_1/w_2 \approx -1.99$ — 실제 비율 -2 에 매우 가까움.
- **검증:** 보지 못한 새 궤적 쌍 3개 — 3/3 정답.

Recovered reward direction in 2D (smoothness, roughness) space



회색 등고선 = 학습된 r_ϕ , 파랑 = r^* 방향 (2, -1), 주황 = 학습된 방향 (7.26, -3.64) — 같은 방향, 길이만 다름.

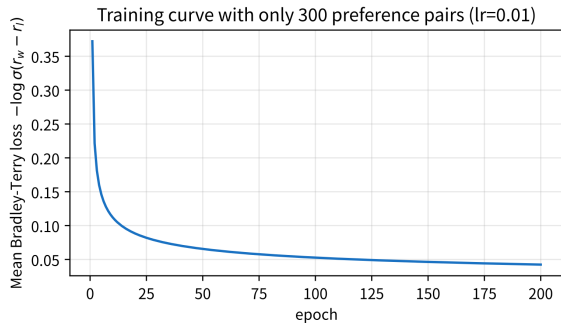
왜 스케일은 안 나오고 방향만 나오는가

- Bradley-Terry 확률은 $r_w - r_l$ 의 **차이**에만 의존 — 모든 가중치를 **같은 배수로** 늘려도 예측 확률과 동일.
- 즉 **스케일 불변**: “이 퀘적의 점수가 몇 점”이라는 **절대 해석 불가**. 오직 **상대 순위**만 의미.
- (LLM 보상모델도 같은 이유에서 절대 점수 해석 불가.)

보상 b 가 학습되지 않는 이유

$r_w - r_l$ 에서 공통 보상 b 는 **완전히 소거됨** $\Rightarrow$ 어떤 값이든 손실을 안 바꿔 **추정 불가능**. 그래서 코드에서 $b = 0$ 고정, w_1, w_2 만 갱신. “차이만 본다”는 설계가 스케일·상수 불변성을 **구조적으로** 만든다.

학습 곡선: “방향”을 빠르게 잡아 버린다



x =에폭, y =에폭당 평균 손실 $-\log \sigma(r_w - r_l)$. 처음 수십
에폭 급락, 이후 거의 수평.

- 300 쌍, 200 에폭: 처음 **급격히** 떨어지다가 어느 시점부터 **수평** — “부드러움이 더 가치 있다”는 **방향**을 빠르게 잡은 것.
- 곡선이 0까지 안 내려가는 이유: **가까운** 궤적 쌍은 진짜 보상에서도 차이가 작아 $\sigma(r_w - r_l)$ 이 1에 완전히 도달 못 함.
- 손실의 **절대값**보다 **하향 추세**를 읽으라.

자주 하는 실수: 절대값을 그대로 해석하기

- 학습된 $(7.264, -3.643)$ 이 진짜 $(2, -1)$ 과 스케일이 다르다 → “모델이 부정확하다”로 오해하기 쉬움.
- Bradley-Terry 손실은 오직 **상대 차이**만 학습 신호로 쓰므로, **절대 스케일은 원래 정해지지 않음**.
- 실전에서 보상모델 출력을 “만족도 점수”처럼 절대적으로 해석하면 안 됨 — 항상 **다른 후보와 비교했을 때** 상대적으로 얼마나 나은가로만 읽어야.

FAQ: 300개는 숫자적으로 특별한가 / tie 처리

쌍 수보다 **쌍이 특징 공간의 서로 다른 방향을 얼마나 대표하느냐**가 핵심 — 30개면 방향이 고르게 안 커버돼 비율이 -2 에서 꽤 벗어날 수 있음. “둘 다 비슷하다”(tie)를 win/lose에 억지로 넣으면 **잡음** — 실전에선 걸러내거나, 연속 선호($0 \sim 1$ 실수값)로 확장.

12.2 정리

- Bradley-Terry 손실(로지스틱회귀와 같은 교차엔트로피)로 사람의 **비교**로부터 보상모델 r_ϕ 학습 → 그 보상을 PPO에 넘겨 정책 최적화(RLHF 두 단계).
- **차이**에만 의존 $\Rightarrow$ 절대 스케일은 정해지지 않고 **방향(상대적 중요도)**만 복원 — 실습: 300 쌍으로 비율 -2 를 -1.99 까지 복원, 검증 3/3.
- 시연은 **상한이 사람**. 비교는 **상한 초월 가능** — 사람이 시연 안 한 새 시도에도 점수를 줄 수 있어서.
- “대리 모델 보상으로의 교체”가 강력한 동시에, **보상 해킹**(Goodhart의 법칙 RL 버전)의 입구.

12.3 모방학습 vs 강화학습: 언제 무엇을 쓰는가

“어떤 게 낫나요?”는 잘못된 질문

	모방학습 (BC/DAGger)	강화학습 (Ch. 2~11)
필요한 것	전문가 시연(또는 비교)	보상함수, 환경과의 상호작용
탐험 필요 여부	기본적으로 불필요(DAGger는 약간)	필수 (탐험-활용 딜레마)
안전성	상대적으로 안전(전문가 행동만 따름)	학습 중 위험한 행동 시도 가능
성능 상한	전문가 수준이 상한	이론상 전문가를 뛰어넘을 수 있음

- 두 접근은 **동일한 자원을 요구하지 않으므로** 비교 대상 자체가 다름.
- 1번째 행(필요한 것)이 사실상 **결정변수** — 나머지 세 행은 그 선택이 만드는 **결과물**.

자원이 선택을 정한다 — 그리고 섞어 쓰기

- **시연이 있는 문제** (시뮬레이터에 잘 동작하는 기존 컨트롤러 포함 — “전문가”는 반드시 사람이 아니어도 됨) → BC 쪽이 이롭다.
- **시연이 없고 보상을 직접 정의할 수 있는 문제** (게임 점수, 에너지 소모량 같은 측정 가능한 목표) → 순수 RL이 자연스럽다.

실전: 둘을 섞어 쓴다

행동 복제로 “**그럴듯한 초기 정책**”을 빠르게 만든 뒤, 그 위에서 RL(PPO 등)로 **미세조정**해 전문가 한계를 넘는다. 모방학습과 선호 기반 모델은 RL 알고리즘을 **대체하는 게 아니라**, 그 알고리즘에 넣을 **데이터와 보상을** 더 안전·저렴하게 얻는 법.

역사적 배경: 이론으로 처음 보여준 2011년 논문

- “시연으로 배운 로봇이 왜 혼자서는 한계에 부딪히나”를 **이론적으로** 처음 보여준 논문: Ross, Gordon & Bagnell (2011) — 12.1절의 DAgger 원 논문.
- 12.1절 “복합 오차”를 **공분포 시프트**로 정식화:
- BC는 **전문가가 방문한** 상태에서 정답만 배우지만, 배포 시 **자기 자신의** 작은 실수로 벗어나면 그 낯선 상태의 오차가 “학습 데이터 분포”에 포함되지 않으므로 **고르게 커진다**.

“시연이 있어도 긴 episode에선 BC만으론 부족하다”는 이 절의 주장은 감이 아니라 2011년 결과의 직접적 귀결.

p^T 문제: “전문가를 못 넘는다”를 긴 episode로

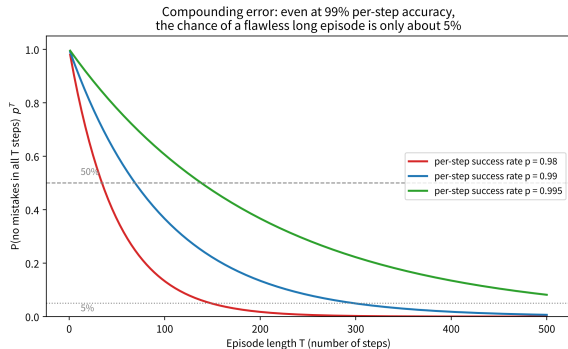
- “모방학습은 전문가 수준이 상한” — **긴 episode** 관점에서 다시 쓰면 전혀 다른 이야기.
- BC를 잘 학습시켜도 배포 단계에서 매 스텝 “전문가가 하듯” 올바른 행동 확률 $p < 1$ — 시연에 없던 미세 차이, 센서 노이즈, 작은 교란.
- 매 스텝 독립이라면:

$$P(\text{오점 없는 episode}) = p^T$$

episode 길이 T	50	100	200	300
$p = 0.980$	0.364	0.133	0.018	0.002
$p = 0.990$	0.605	0.366	0.134	0.049
$p = 0.995$	0.778	0.606	0.367	0.222

$p = 0.99$ 라도 $T = 300$ 에서 “한 번도 삐끗 안 함”은 4.9%. “매 스텝 잘한다”가 “긴 episode를 완주한다”를 보장하지 못함.

지수 감쇠: DAgger 없이 BC만 쓰는 정량적 비용



$p \in \{0.98, 0.99, 0.995\}$ 의 p^T 곡선.

그리고 그 위에서 **RL로 미세조정**하면 그 상한을 넘을 수 있을까 — 다음 실험에서 직접 확인.

- $p = 0.995$ 이면 같은 300스텝에서 **22%까지** 살아남음.
- **1스텝 성공률 0.5%** p 가 긴 episode에서 “거의 불가능”과 “가끔 된다”를 가른다.
- DAgger가 “시연에 없던 상태로 비틀어진” 경로의 상태까지 학습 데이터에 포함시켜 **p 자체를 높이는** 이유가 바로 이것.
- 실습: p 와 T 를 바꿔 p^T 곡선을 직접 그려보기.

실습: 5×5 격자, 세 개를 붙여 놓고 달리기

- **환경**: 5×5 격자. 시작 $(4, 0) \rightarrow$ 목표 $(0, 4)$. 매 스텝 보상 $-1 \Rightarrow$ **가장 짧은 경로** = 보상 합이 가장 큰 경로.
- **전문가**: 안전하지만 **비최적** 우회 **10스텝** (보상 -10). 최적은 **8스텝** (-8) — **전문가가 최적을 모르고** 가는 상황.
- **위험**: 한가운데 $(2, 2)$ 에 **함정** — 진입 시 -50 , episode 종료. 시연 경로엔 함정이 없으나, **5% 확률 교란**으로 이동이 명령과 무관한 인접 칸으로 바뀐다.
- **500 episode**, seed 고정.

세 그룹: A(순수 BC) · B(순수 RL) · C(BC+RL)

A	순수 BC — 시연 (s, a)로만 학습. 시연에 없던 상태에서는 무작위 .
B	순수 RL — Q-learning(Ch. 6, DQN의 핵심)을 0부터 ϵ -탐험으로 학습.
C	BC+RL — BC 정책의 가치함수를 초기값 으로 주고, 그 위에 Q-learning 미세조정.

BC+RL의 “초기값”이 정확히 무엇인가

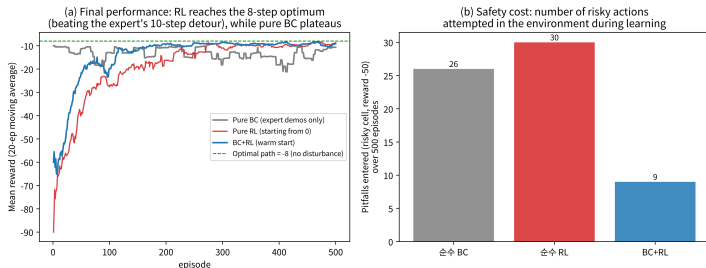
BC가 학습한 것은 **정책** π 이지 가치함수 아님. 이 실험의 초기값: 시연 경로 위 상태 s 에서 전문가 행동을 따르면 남은 스텝 수 k 만큼 더 받아 $Q(s, \pi(s)) \approx -k$, 다른 행동은 $-(k-1)$, 시연에 없는 상태는 0(미학습)
— “**시연 근방에선 시연만큼 정확하고, 바깥은 아직 모름.**”

결과 1: 세 그룹의 숫자 (seed 0 고정)

	최종 20ep 평균 보상	함정 진입 (500ep)
A: 순수 BC	− 10.65	26회
B: 순수 RL	− 8.70	30회
C: BC+RL	− 9.15	9회
전문가 우회	−10 (10스텝)	(시연엔 함정 없음)
최적 경로	−8 (8스텝)	

- **A:** 단 한 번도 전문가(−10)를 넘는 episode가 **없었다** — “시연은 상한이 사람”이 숫자로 확인.
- **B:** 8스텝 최적 근접, 전문가 10스텝 **추월**.
- **C:** 완전 최적은 못 미치나, 추월 **193ep** (B는 273ep), 함정 **9회**로 가장 적음.

결과 2: 그림이 보여주는 것



(a) episode별 평균 보상(20-ep 이동평균), (b) 500ep 함정 진입 횟수 —
BC 26 / RL 30 / BC+RL 9.

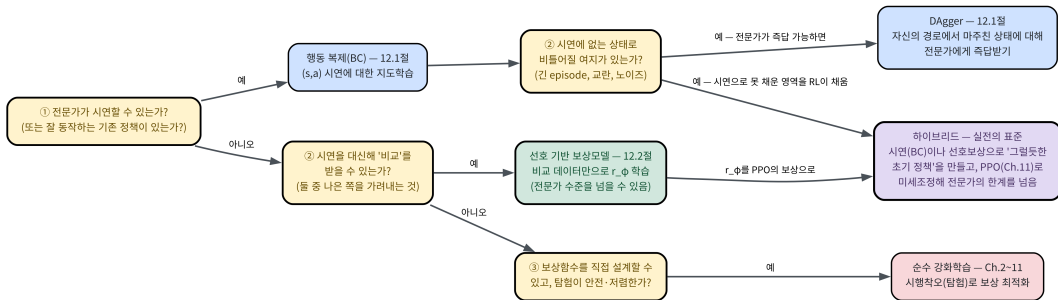
- B: 특히 학습 초기 50ep에 20회 — “아직 배우는 중” 에이전트가 위험 행동을 가장 많이 시도.
- C: 초기 안전성(9 vs 26/30)과 추월 속도 (193 vs 273)를 동시에 획득.

한 그림에 압축된 핵심

순수 BC = 안전하지만 못 넘고, 순수 RL = 넘을 수 있지만 위험 행동을 많이 하고, BC+RL = 둘의 장점을 동시에.

선택 흐름: “무엇을 가졌는가”를 확인하며

시연·비교(모방학습)와 보상·상호작용(강화학습) — 무엇으로 배울 것인가
'가진 자원'(시연, 비교, 보상함수, 안전한 상호작용)을 확인하며 결정



(1) 시연이 있는가 → 예: BC → (시연 밖 상태로 비틀어질 여지가 있으면 DAgger 또는 BC+RL). (2) 시연 없으면 비교를 받을 수 있는가 → 예: 선호 기반 보상모델(12.2) → PPO 미세조정. (3) 둘 다 없으면 보상함수 직접 설계 + **안전한** 탐색이 가능한가 → 예: 순수 RL.

실전 표준: 초기 정책 + PPO 미세조정

- 흐름의 마지막 칸 — “시연(또는 비교)으로 **초기 정책**을 만들고 **PPO로 미세조정**” — 이 **실전 표준**.
- LLM의 RLHF(12.2)가 공식적으로는 이 구조: 사전훈련 언어모델(= 시연에서 만든 초기 정책) → 사람 비교로 학습한 보상모델로 PPO → **사람이 시연한 적 없는** 새 표현 생성.
- “초기 정책이 이미 그럴듯해서 PPO가 근방만 탐색하면 된다” — 이것이 12.2 실습의 “비교 300개로 방향 복원”을 가능하게 한 이유이기도.
- Ch. 13 로봇 제어: BC로 초기 안정화(서 있는 자세) → 보상모델로 미세조정.

자주 하는 실수: “시연이 있으니”를 “충분하다”로

- 12.1 복합 오차 + 이번 실험 (b)를 연결하면, 가장 흔한 실무 오류: **“시연 데이터가 있으니 BC로 끝난다”**.
- 시연이 **있는 것** $\neq$ 시연이 **충분한 것**.
- 순수 BC가 500ep 중 26번 함정 진입한 이유: 시연에 없던 (교란으로 밀려난) 상태에서 **무작위**로 행동했기 때문.
- 이 “시연 밖 상태” 비중이 클수록(긴 episode / 교란 큰 환경 / 전문가가 **비최적** 경로 시연) BC 단독 한계 커짐.

판정 기준

“배포 시 **시연에 없던 상태**를 방문할 확률이 몇 %인가?” 0에 가까우면 BC로 충분. 아니면 **Dagger(12.1)** 또는 **BC+RL**(그룹 C)이 필요.

FAQ: 상한, ML1의 RLHF, 26회, BC+RL의 한계

- **Q: DAgger도 상한=전문가?** A: 그렇다 — 복합 오차만 줄여 “전문가 수준에 더 **안정적으로** 도달”할 뿐. 반면 선호 기반은 **비교**만 받으므로 정답 밖 시도에도 점수를 줘 **초월 가능** — 모방학습보다 RL에 가까움.
- **Q: ML1(Ch. 17)의 LLM RLHF와 같은 구조?** A: 수학(BT 모델, 보상모델, RL 최적화)은 **동일** — 정책이 생성하는 것(토큰 vs 관절 토크)만 다름.
- **Q: BC가 함정 26번 = “BC가 위험하다”?** A: 아님 — **정책 자체**가 위험 행동을 고른 게 아니라, **시연 커버리지 밖에** 정책이 없었기 때문. “BC는 가장 안전하다”는 주장은 **시연이 배포 분포를 커버할 때만** 성립.
- **Q: 왜 RL을 본격적으로 돌려야 하나?** A: BC 초기값은 시연 **근방**만 정확하고, 바깥 Q값은 0 — 그 영역을 채우려면 **실제로 방문**해야 하므로 순수 RL과 같은 탐험이 필요. 그래서 최종 -9.15에 머물고, 충분히 탐색한 후에야 8스텝 최적에 수렴.

12.3 정리: 자원이 정한다

- **시연이 있으면** BC로 가장 안전·빠르게 시작. 긴 episode 또는 시연 밖 상태로 비틀어질 가능성 있으면 p^T 지수 감쇠가 BC의 **실질 한계** — DAgger로 시연을 자기 경로에 맞게 보강하거나 RL로 미세조정.
- **시연 없이 “비교”만 받으면**(12.2) 선호 기반 보상모델로 사람이 보상을 직접 쓰지 않고도 **전문가를 넘을 수 있는** 보상을 만들고 PPO로 최적화.
- **둘 다 없으면** Ch. 2~11의 순수 RL.
- **실전 표준** = 시연(또는 비교)으로 초기 정책 → RL로 한계를 넘게 하는 **하이브리드**.
- 숫자로: 초기 안전성 **9 vs 26/30**, 최종 성능 (추월) **193 vs 273** episode.

12주차 정리

12.1 시연에서

- BC = 시연 (s, a) 쌍을 라벨로 쓴 지도학습(로지스틱 회귀 코드 그대로).
- 실패는 **분포** 문제 (공분포 시프트, 복합 오차) — p^T 지수 감쇠 / H^3 세제공.
- DAgger = “에이전트가 달리고, 전문가가 라벨만” 반복 — 보장은 완화이지 해결 아님.

12.2 비교에서

- Bradley-Terry(차이만 보아 스케일 불변)로 보상모델 학습 → PPO 보상으로(RLHF).
- 시연 = 상한이 사람, 비교 = **상한 초월 가능**.
- 대신 **보상 해킹** 위험(Goodhart).

12.3 무엇을 쓸 것인가

- 선택은 **가진 자원**(시연/비교/보상함수)이 정함.

핵심 한 줄

사람이 주는 것이 다르면 알고리즘이 달라진다 — **정답 행동** → DAgger(상한=사람), **비교** → 보상모델 + PPO(초월 가능), **보상함수** → 순수 RL.

다음 주 — Chapter 13. 로봇 시뮬레이션과 제어 기초

이 장의 방법들을 실제 로봇 시뮬레이션 환경으로 옮겨, **시연으로 만든 초기 정책**을 PPO가 어떻게 **미세조정**하는지, 그리고 보상 해킹이 실제 로봇에서 어떻게 관찰되는지 구체적으로 본다.